이론 4차시

하네스: AI를 진짜로 일하게 만드는 소프트웨어

학습 목표

- ✓ 하네스가 모델과 제품의 차이를 만드는 이유를 설명할 수 있어요
- ✓ CLAUDE.md의 구조와 역할을 설계할 수 있어요
- ✓ 에이전트 루프(Gather/Act/Verify/Repeat)의 각 단계를 비유로 설명할 수 있어요

? 핵심 질문

같은 AI 모델인데 어떤 제품은 채팅봇이고, 어떤 제품은 자율 에이전트예요. 이 차이는 모델의 차이일까요? 아니면 모델을 감싸는 **소프트웨어**의 차이일까요?

1. Product Overhang: 능력은 있는데 제품이 없었다

세계에서 가장 똑똑한 모델이 가장 무능해지는 순간

Claude.ai 채팅 인터페이스에서 **이 프로젝트의 auth 모듈을 리팩토링해줘**라고 말해보세요. 돌아오는 답은 **프로젝트의 코드를 보여주시면 도와드리겠습니다**예요. 파일을 읽을 수 없고, 코드를 실행할 수 없고, 결과를 검증할 수 없어요. 세계에서 가장 똑똑한 모델이 아무것도 못 하는 상태예요.

같은 말을 Claude Code에서 해보세요. Claude Code는 프로젝트 구조를 탐색하고(Glob, Grep), auth 관련 파일을 읽고(Read), 코드를 수정하고(Edit), 테스트를 실행하고(Bash), 실패하면 수정하고 다시 테스트해요. 같은 모델이에요. Claude Opus 4.6. 달라진 것은 모델을 감싸는 소프트웨어, 즉 **하네스(Harness)**예요.

비유를 들어볼게요. 세계 최고의 외과 의사가 있다고 상상해보세요. 이 의사에게 수술실도, 메스도, 모니터도 주지 않고 빈 방에 세워놓으면 어떻게 될까요? 환자를 볼 수 없고, 진단할 도구가 없고, 수술할 장비가 없어요. 아무리 뛰어난 의사라도 할 수 있는 것이 없어요. 하네스는 이 의사에게 수술실, 도구, 모니터를 갖춰주는 것과 같아요.

같은 모델, 다른 제품

Boris Cherny(Anthropic, Claude Code 초기 개발자)가 이 현상을 **Product Overhang**이라 불렀어요. 모델의 능력은 이미 충분한데, 그 능력을 제품으로 만드는 소프트웨어가 없었다는 뜻이에요. Cherny가 처음 Claude에게 파일시스템 접근 권한을 줬을 때 **Claude가 파일시스템을 탐색하는 것이 경이로웠다**고 회고한 바 있어요. 파일을 읽게 해줬더니 모든 것이 달라진 거예요.

이것은 모델이 갑자기 똑똑해진 것이 아니에요. 모델은 그대로인데 **접근할 수 있는 것**이 달라진 거예요. 눈가리개를 벗겨준 것과 같아요.

두 제품의 차이를 직접 비교해볼게요.

항목	Claude.ai (채팅)	Claude Code (에이전트)
모델	Claude Opus 4.6	Claude Opus 4.6
파일 접근	불가능 (사용자가 복사해야 함)	자동으로 탐색, 읽기, 수정
코드 실행	불가능	Bash 도구로 직접 실행
검증	불가능 (사용자가 확인)	테스트 자동 실행, 실패 시 재수정
루프	사람이 수동으로 반복	소프트웨어가 자동 반복
작업 시간	30분 (대화 10회)	3분 (자동 루프)

> 채팅봇과 에이전트의 차이는 모델의 차이가 아니라 하네스의 차이예요. Product Overhang이란, 모델 능력은 이미 있는데 그것을 꺼내 쓸 소프트웨어가 없는 상태를 말해요.

2. 하네스의 두 가지 역할

Simon Willison이 2026년 3월에 정의한 것처럼, 하네스(Harness)란 **LLM을 보이지 않는 프롬프트와 호출 가능한 도구로 확장하는 소프트웨어**예요. 이 정의에 하네스의 두 가지 역할이 모두 담겨 있어요.

역할 1: 보이지 않는 프롬프트를 관리해요

사용자가 Claude Code에서 **이 파일 고쳐줘**라고 한 줄을 치면, 실제로 모델에게 전달되는 것은 그 한 마디가 아니에요. 시스템 프롬프트 약 5,000 토큰, CLAUDE.md 내용, 도구 정의 목록, 이전 대화 기록이 앞에 붙어요. 사용자는 한 줄을 썼지만 모델은 수만 토큰을 받은 셈이에요.

비유를 해볼게요. 병원에 가서 의사에게 **배가 아파요**라고 한 마디를 하죠. 그런데 의사가 보는 것은 그 말만이 아니에요. 차트에는 과거 진료 기록, 알레르기 정보, 복용 중인 약, 최근 검사 결과가 다 적혀 있어요. 환자는 한 마디를 했지만, 의사는 수십 페이지의 맥락 위에서 그 한 마디를 해석해요. 하네스가 하는 일이 바로 이거예요. 사용자의 한 마디에 풍부한 맥락을 붙여서 모델에게 전달하는 거예요.

이 보이지 않는 부분에는 다음이 포함돼요.

- **시스템 프롬프트**: 모델의 행동 규칙, 응답 형식, 제약 조건
- **CLAUDE.md**: 프로젝트별 설정 (빌드 명령어, 코드 스타일, 디렉토리 구조)
- **도구 정의**: 사용 가능한 도구 목록과 각 도구의 사용법
- **대화 이력**: 이전 턴의 전체 내용 (LLM은 무상태이므로 매번 재전송)

역할 2: 호출 가능한 도구를 제공해요

모델이 외부 세계와 상호작용하는 통로예요. 모델이 **auth.ts 파일을 읽어야겠다**고 판단하면 Read 도구를 호출하고, 하네스가 실제로 파일을 읽어서 결과를 컨텍스트에 넣어요. 수정이 필요하면 Edit 도구를 호출하고, 테스트가 필요하면 Bash 도구를 호출해요.

다시 병원 비유로 돌아가면, 의사가 **혈액 검사를 해주세요**라고 간호사에게 지시하는 것과 같아요. 의사가 직접 피를 뽑는 것이 아니라, 지시를 내리면 간호사(하네스)가 실행하고 결과를 가져다줘요. 모델이 **이 도구를 이 인자로 호출해줘**라고 요청하면, 하네스가 실제로 실행하고 결과를 돌려주는 거예요.

핵심은 LLM이 무상태(stateless)라는 점이에요. Willison이 지적한 것처럼, **LLM은 매번 실행될 때마다 같은 백지에서 시작**해요. 대화가 이어지는 것처럼 보이지만 실제로는 하네스가 매번 전체 대화 이력을 다시 전송해요. 토큰 캐싱이 비용을 줄여주긴 하지만, 구조적으로 모든 턴마다 전체 컨텍스트가 재구성돼요. 이 재구성을 자동으로 처리하는 것이 하네스의 일이에요.

> 하네스의 두 가지 역할: (1) 보이지 않는 프롬프트 관리 = 사용자의 한 마디에 풍부한 맥락을 붙이기. (2) 호출 가능한 도구 제공 = 모델이 외부 세계와 상호작용할 수 있게 하기. 이 두 가지가 합쳐지면 채팅봇이 에이전트가 돼요.

3. CLAUDE.md: 에이전트의 두뇌를 설계하는 방법

보이지 않는 프롬프트 중 가장 중요한 것이 CLAUDE.md예요. 프로젝트 루트에 놓는 파일 하나가 에이전트의 행동 전체를 바꿔요. CLAUDE.md는 하네스가 매 턴마다 자동으로 읽어서 시스템 프롬프트에 주입하는 프로젝트별 설정 파일이에요.

나쁜 CLAUDE.md vs 좋은 CLAUDE.md

비유를 먼저 할게요. 새로 온 인턴에게 업무를 맡긴다고 생각해보세요.

나쁜 지시: **일 잘 해주세요. 꼼꼼하게요.**

이건 아무 정보도 주지 않아요. 좋은 일이 뭔지, 꼼꼼하다는 게 어떤 기준인지 인턴은 알 수 없어요.

좋은 지시: **월요일 아침 9시까지 주간 보고서를 작성해주세요. 형식은 공유 폴더의 템플릿을 따르고, 숫자는 ERP에서 뽑아주세요. 완성되면 팀장님 메일로 보내기 전에 저한테 먼저 확인받아주세요.**

구체적이고, 실행 가능하고, 검증 기준이 있어요. CLAUDE.md도 같은 원리예요.

나쁜 CLAUDE.md	좋은 CLAUDE.md
코드를 잘 짜주세요. 테스트도 꼼꼼히 하고 좋은 코드를 작성하세요. 사용자 경험을 중시합니다.	**빌드/테스트 명령어**: npm run build, npm test, 단일 테스트는 npm test – –grep "test name". **코드 스타일**: TypeScript strict mode, 함수형 우선, any 금지. **커밋 규칙**: conventional commits (feat:, fix:). **프로젝트 구조**: src/api/ (라우트), src/domain/ (비즈니스 로직), src/infra/ (DB).

좋은 CLAUDE.md의 네 가지 구성 요소

좋은 CLAUDE.md에는 정확히 네 가지가 들어가요. 이 네 가지가 빠지면 에이전트는 프로젝트를 탐색하는 데 컨텍스트의 절반을 낭비해요.

구성 요소	역할	예시
빌드/테스트 명령어	에이전트가 코드를 실행하고 검증하는 방법	npm run build, pytest -v
코드 스타일 규칙	일관된 코드를 작성하는 기준	TypeScript strict, any 금지, Result 타입
디렉토리 구조	어떤 파일이 어디에 있는지의 지도	src/api/, src/domain/, src/infra/
커밋 규칙	변경사항을 기록하는 형식	conventional commits, 커밋 전 테스트 통과 필수

에이전트가 첫 턴에서 **이 프로젝트는 어떻게 돌아가는가**를 즉시 파악할 수 있어야 해요. CLAUDE.md가 잘 쓰여 있으면 에이전트가 프로젝트 탐색에 쓰는 턴 수가 줄어들고, 그만큼 실제 작업에 더 많은 컨텍스트 예산을 쓸 수 있어요.

핵심은 이거예요. CLAUDE.md는 **사람이 읽는 문서**가 아니라 **에이전트가 실행하는 지침**이에요. README는 사람을 위한 것이고, CLAUDE.md는 AI를 위한 거예요. Simon Willison의 관련 개념인 AGENTS.md도 같은 원리예요. 에이전트가 시행착오로 알아내야 할 것을 문서로 미리 알려주면 루프가 빨라져요.

비개발자를 위한 CLAUDE.md 이해

개발을 하지 않는 분도 이 원리를 이해할 수 있어요. CLAUDE.md는 결국 **새로 온 사람에게 주는 업무 매뉴얼**이에요.

새 직원이 첫날 출근했다고 생각해보세요. 이 직원에게 필요한 정보가 네 가지 있어요.

- **어떻게 확인하나요?**: 결과물을 제출하기 전에 어떤 절차를 거쳐야 하는지 (빌드/테스트)
- **어떤 규칙을 따르나요?**: 보고서 형식, 언어 스타일, 브랜드 가이드라인 (코드 스타일)
- **뭐가 어디에 있나요?**: 공유 폴더 구조, 담당자 연락처, 참고 자료 위치 (디렉토리 구조)
- **완료 보고는 어떻게 하나요?**: 일일 보고 양식, 승인 절차 (커밋 규칙)

이 네 가지를 미리 정리해두면 새 직원이 첫날부터 일할 수 있어요. 정리하지 않으면 첫 주를 질문하는 데 다 보내요. AI 에이전트도 정확히 같아요.

> CLAUDE.md는 에이전트의 업무 매뉴얼이에요. 빌드/테스트 방법, 코드 스타일, 디렉토리 구조, 커밋 규칙 네 가지가 핵심이에요. 잘 쓰면 에이전트가 첫 턴부터 일하고, 못 쓰면 에이전트가 컨텍스트 절반을 탐색에 낭비해요.

4. 에이전트 루프: 수집, 행동, 검증, 반복

하네스의 심장은 루프예요. 이 루프의 원형은 Yao 등(2022)의 ReAct 패턴이에요. **추론 흔적(reasoning trace)**과 **행동(action)**을 교차시키는 거예요. 모델이 **무엇을 해야 하는지** 추론하고, 그에 따라 행동하고, 결과를 관찰하고, 다시 추론해요. Anthropic의 Thariq Shihipar(2025)가 이 원형을 실전에 맞게 재정리한 패턴이 표준이 됐어요.

Gather context, Take action, Verify work, Repeat.

의사의 진료 과정으로 이해하기

에이전트 루프는 의사의 진료 과정과 정확히 같아요.

단계	에이전트	의사	도구
Gather (수집)	파일 탐색, 코드 검색	문진, 검사 결과 확인	grep, Read, Glob
Think (추론)	원인 분석	진단	(모델 내부)

단계	에이전트	의사	도구
Act (행동)	코드 수정	처방, 시술	Edit, Bash
Verify (검증)	테스트 실행	경과 관찰	Bash (test)
Repeat (반복)	실패 시 다시 수집부터	호전 없으면 재검사	루프 계속

의사가 한 번의 진찰로 모든 병을 알아내지 못하는 것처럼, 에이전트도 한 번에 완벽한 답을 내지 못해요. 수집-행동-검증을 반복하면서 점점 정답에 가까워지는 거예요. 이것이 에이전트 루프의 핵심이에요.

[도표 4-1] 워크플로우와 에이전트의 차이. 워크플로우는 경로가 사전에 고정돼 있고, 에이전트는 모델이 상황에 따라 다음 행동을 스스로 결정해요.

실전 시나리오: 로그인 버그 수정

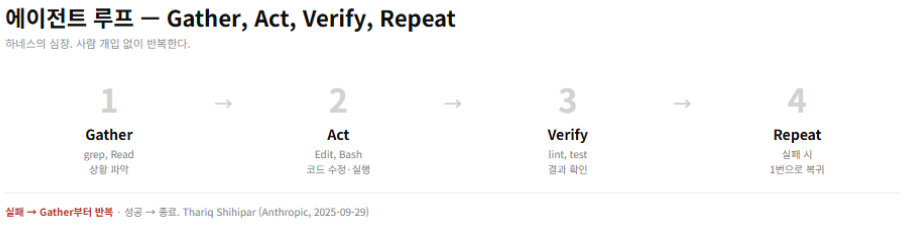
이 루프가 실전에서 어떻게 돌아가는지 따라가볼게요. **로그인이 안 된다는 버그 리포트**가 들어왔어요.

단계	행동	내용
Gather	grep, Read	src/ 에서 login 키워드 검색. 관련 파일 5개 발견. login.ts 코드 확인
Think	추론	42번 줄에서 토큰 만료 체크가 빠져 있음을 발견
Act	Edit	login.ts 42번 줄 수정
Verify	Bash	npm test auth.test.ts 실행. 1건 실패

단계	행동	내용
Think	추론	타임존 문제 파악. UTC 변환 필요
Act	Edit	login.ts 재수정
Verify	Bash	auth 테스트 통과. 전체 테스트 통과

이 전체 과정에서 사람이 개입한 것은 처음 **로그인 버그 고쳐줘** 한 마디뿐이에요. 나머지는 하네스의 에이전트 루프가 자동으로 돌린 거예요. 수집(grep, Read), 행동(Edit), 검증(Bash: test), 실패 시 다시 수집부터.

여기서 가장 중요한 단계가 뭘까요? **검증(Verify)**이에요. 검증이 빠지면 에이전트는 **고쳤다**고 보고하면서 실제로는 망가뜨린 상태가 돼요. 의사가 약을 처방하고 경과를 확인하지 않는 것과 같아요.



[도표 4-2] 에이전트 루프의 구조. Gather, Act, Verify, Repeat의 순환이 하네스의 심장이에요.

5. ACI: 도구 설계가 프롬프트보다 중요해요

에이전트에게 도구를 주는 것만으로는 부족해요. 도구를 **어떻게 정의하느냐**가 결과를 결정해요.

절대경로 하나가 성패를 갈랐어요

Anthropic의 Erik Schluntz와 Barry Zhang(2024)이 SWE-bench라는 코딩 벤치마크를 풀면서 발견한 것이 있어요. **프롬프트보다 도구 최적화에 더 많은 시간을 썼다**고 해요. 구체적으로, 파일 편집 도구에서 **상대 경로를 절대 경로로 바꾸는 것만으로** 도구 사용 오류가 완전히 사라졌어요.

왜 그런지 비유로 설명할게요. 누군가에게 **3층 회의실로 가세요**라고 말하면, 이 사람이 지금 어느 건물에 있느냐에 따라 전혀 다른 곳에 갈 수 있어요. 하지만 **서울시 강남구 OO빌딩 3층 301호 회의실**이라고 말하면 어디에 있든 정확히 도착해요. 상대 경로는 **3층 회의실**이고, 절대 경로는 **전체 주소**예요.

에이전트가 디렉토리를 이동한 후에 **auth.ts**를 수정하면 잘못된 위치의 파일을 수정하거나 파일을 아예 못 찾아요. /home/user/project/src/auth/auth.ts라고 절대 경로로 명시하면 이 문제가 사라져요. 사소해 보이는 변경이 전체 작업의 성패를 갈랐어요.

ACI 설계의 세 가지 원칙

Anthropic은 이 원칙을 ACI(Agent-Computer Interface) 디자인이라 불러요. 인간-컴퓨터 인터페이스(HCI)에 들이는 노력만큼을 에이전트-컴퓨터 인터페이스에도 들여야 한다는 거예요.

비유를 하나 더 할게요. 여러분이 처음 보는 가전제품을 사용한다고 생각해보세요. 설명서가 없거나, 버튼에 아무 표시가 없으면 사용하기 어렵죠. 반대로 버튼마다 명확한 라벨이 있고, 잘못된 조작을 하면 경고가 뜨면 실수가 줄어요. ACI 설계도 같은 원리예요. 에이전트가 도구를 처음 보는 사용자라고 생각하고 설계하는 거예요.

원칙	뜻	실천 방법
모델의 입장에서 생각할 것	모델이 도구를 처음 보는 주니어 개발자라고 상상해요. 암묵적 규칙은 없어요	파라미터 이름, 반환값 형식, 에러 시 행동을 명시해요
주니어에게 설명하듯 문서를 쓸 것	모호한 설명은 모호한 결과를 낳아요	도구 설명은 길어도 괜찮아요. 수백 줄이 될 수도 있어요
실수가 어렵도록 설계할 것	잘못 쓸 수 있는 설계는 반드시 잘못 쓰여요	상대 경로 대신 절대 경로 강제. 위험한 기본값 제거

도구 정의 자체가 컨텍스트 윈도우의 예산을 잡아먹는다는 점도 중요해요. Claude Code의 시스템 프롬프트에는 도구 정의만으로 수천 토큰이 들어가요. 도구가 많을수록 컨텍스트가 줄어들어요. 그래서 도구는 적게, 하지만 정확하게 설계하는 것이 원칙이에요.

Claude 4에서 관찰된 현상이 이 원칙을 뒷받침해요. 멀티에이전트 시스템에서 Claude 4가 자기 도구 설명을 자율적으로 재작성한 결과, 태스크 완료 시간이 40% 감소했어요. 도구 설명이 곧 성능이라는 증거예요.

> 나쁜 프롬프트보다 나쁜 도구 설명이 더 치명적이에요. 프롬프트는 한 번 읽히지만, 도구 정의는 매 턴마다 컨텍스트에 포함되기 때문이에요. ACI 설계란, 에이전트가 도구를 처음 보는 사람처럼 명확하고, 실수하기 어렵게 도구를 설계하는 것이에요.

6. 검증 3단계: 에이전트를 믿되 확인해요

에이전트 루프에서 검증이 빠지면 치명적이에요. 검증 없는 에이전트는 **완료했습니다**라고 보고하면서 실제로는 문제를 더 키운 상태가 될 수 있어요. Anthropic이 제시한 검증 3단계가 있어요.

3단계의 비유: 시험 채점

비유로 먼저 이해해볼게요. 학생이 시험을 봤다고 생각해보세요.

- **1단계 (규칙 기반)**: OMR 카드를 기계로 채점하는 거예요. 빠르고, 저렴하고, 자동화가 돼요. 하지만 서술형 문제는 채점할 수 없어요.
- **2단계 (시각 피드백)**: 선생님이 답안지를 직접 보는 거예요. 글씨가 읽기 쉬운지, 그림이 맞는지 눈으로 확인해요. 기계보다 느리지만 더 많은 것을 볼 수 있어요.
- **3단계 (LLM-as-Judge)**: 다른 전문가에게 답안을 보여주고 **이 답이 맞나요?**라고 묻는 거예요. 가장 비싸지만 복잡한 판단이 가능해요.

단계	방법	장점	한계
1단계	규칙 기반 (lint, type check, test)	가장 빠르고 저렴. 자동화 용이	정해진 규칙만 검사 가능
2단계	시각적 피드백 (브라우저 스크린샷)	사람이 보는 것과 같은 확인	설정 비용이 있음
3단계	LLM-as-Judge (다른 AI가 판정)	가장 유연. 복잡한 판단 가능	가장 비싸고 판정 자체가 틀릴 수 있음

실전에서는 이 3단계를 순서대로 적용해요. 1단계로 거를 수 있는 것은 1단계에서 거르고, 1단계로 안 되는 것만 2단계로 올리고, 그래도 안 되는 것만 3단계로 올려요. 이렇게 하면 비용과 정확성의 균형을 맞출 수 있어요.

장기 작업에서는 2단계의 가치가 특히 커요. Justin Young(Anthropic, 2025)의 실험에서 Puppeteer MCP로 브라우저 자동화 도구를 제공하자 에이전트의 작업 품질이 크게 향상됐어요. Playwright 스크린샷으로 실제 화면을 캡처해서 모델에게 **이게 맞는 결과인가**를 판단시키는 방식이에요. 비용이 들지만, 인간이 보는 것과 가장 가까운 검증이에요.

> 검증 3단계: (1) 규칙 기반 = OMR 채점. (2) 시각 피드백 = 선생님이 직접 보기. (3) LLM-as-Judge = 전문가에게 묻기. 1단계부터 적용하고, 필요한 경우에만 상위 단계로 올려요.

7. 서브에이전트: 컨텍스트를 분리하는 기술

왜 서브에이전트가 필요한가요

하나의 에이전트가 모든 것을 하면 컨텍스트가 빠르게 소진돼요. 10개 파일을 읽고, 3개를 수정하고, 테스트를 돌리면 컨텍스트 윈도우의 상당 부분이 이미 찬 상태예요. 이때 추가 작업을 맡기면 앞서 읽은 파일의 내용이 요약되면서 세부 정보가 사라져요.

비유를 할게요. 책상 위에 서류를 쌓아가면서 일한다고 생각해보세요. 처음엔 서류 5장으로 시작했는데, 일하다 보니 서류가 50장이 됐어요. 책상이 좁아서 아래쪽 서류는 더 이상 보이지 않아요. 이때 해결책은 간단해요. 새 책상을 하나 더 놓고, 특정 업무는 그 책상에서 따로 처리하는 거예요. 끝나면 결과만 가져오면 돼요.

서브에이전트의 작동 방식

서브에이전트는 이 해결책의 소프트웨어 버전이에요. 메인 에이전트가 특정 작업을 격리된 컨텍스트 창에서 실행되는 서브에이전트에게 위임하는 패턴이에요.

메인 에이전트가 **이 파일의 타입 에러를 모두 수정해**라는 작업을 서브에이전트에게 넘겨요. 서브에이전트는 자기만의 컨텍스트 윈도우에서 해당 파일을 읽고, 수정하고, 검증해요. 완료되면 결과 요약만 메인 에이전트에게 돌려줘요. 메인 에이전트의 컨텍스트에는 서브에이전트가 읽은 파일 전체가 아니라 **타입 에러 5건 수정 완료**라는 요약만 들어가요.

Claude Code에서 이 기능은 3일 만에 구축됐고, 개발 과정에서 2일치 작업이 폐기됐어요(Boris Cherny, 2025). 빠르게 시도하고 빠르게 버리는 개발 방식의 전형이에요.

서브에이전트를 쓸 때 vs 안 쓸 때

서브에이전트가 유효한 경우	서브에이전트가 불필요한 경우
독립적인 파일이나 모듈 단위 작업	파일 간 의존성이 높아 전체 맥락이 필요한 작업
탐색적 작업 (여러 디렉토리에서 정보 수집)	한 파일 안에서의 단순 수정
병렬 처리가 가능한 반복 작업	사용자와의 대화 맥락이 중요한 작업

흥미로운 역설이 하나 있어요. 장기 실행 에이전트에서 메인 에이전트가 3시간 동안 작업하면서 컨텍스트가 누적되면, 새 작업을 깨끗한 컨텍스트에서 시작하는 서브에이전트가 오히려 더 정확해요. 오

래된 컨텍스트에 묻혀 있는 메인 에이전트보다 방금 시작한 서브에이전트가 단일 작업에서는 더 나은 결과를 내는 역설이 발생해요.

Anthropic의 Building Effective Agents 문서에서 제시한 **Orchestrator-Workers** 패턴이 이 구조의 일반형이에요. 중앙 LLM이 작업을 동적으로 분해하고, 워커 LLM에게 위임하고, 결과를 합성해요. 서브에이전트는 이 패턴의 코딩 에이전트 구현이에요.

> 서브에이전트는 컨텍스트 소진 문제의 해법이에요. 메인 에이전트가 모든 것을 하는 대신, 독립적인 작업을 별도의 깨끗한 컨텍스트에서 처리하고 결과만 가져와요. 책상이 가득 차면 새 책상을 쓰는 원리예요.

8. Claude Code의 숫자들: 하네스가 만드는 차이

Claude Code가 하네스의 대표 구현이라고 했어요. 숫자로 보면 이 하네스가 얼마나 강력한지 실감할 수 있어요. The Pragmatic Engineer의 Gergely Orosz가 Boris Cherny, Sid Bidasaria(창립 엔지니어), Cat Wu(PM)를 인터뷰한 기사에서 공개된 수치예요.

지표	수치
연간 매출 런레이트	5억 달러 이상 (2025년 5월 일반 출시 이후)
성장률	3개월 만에 사용량 10배
내부 도입 1일차	Anthropic 엔지니어의 20%
내부 도입 5일차	50%
자체 코드 비율	Claude Code의 약 90%가 Claude Code로 작성
릴리즈 속도	내부 60 100회/일, 외부 1회/일
엔지니어당 PR	하루 약 5개

이 숫자들이 의미하는 것을 하나씩 풀어볼게요.

5억 달러 매출은 하네스가 단순한 실험이 아니라 실제로 돈을 버는 제품이라는 뜻이에요. 2025년 5월에 출시해서 불과 몇 달 만에 이 규모에 도달했어요.

엔지니어의 50%가 5일 만에 도입했다는 것은, 한번 써본 사람이 다시 안 쓰지 못한다는 뜻이에요. 1일차에 20%가 시도했고, 써본 사람이 동료에게 퍼뜨려서 5일차에 절반이 쓰고 있었어요.

코드의 90%를 자기 자신으로 작성했다는 것은 특히 주목할 만해요. Claude Code가 Claude Code를 만든다는 뜻이에요. 이것이 가능한 이유가 있어요. 기술 스택을 TypeScript, React with Ink, Bun으로 선택했는데, 이것들은 Claude가 인터넷에서 가장 많이 본 기술이에요. 모델이 잘 아는 언어로 하네스를 만들면 모델이 자기 하네스를 개선할 수 있어요.

엔지니어당 하루 5개의 PR(Pull Request)이라는 숫자는 생산성의 변화를 보여줘요. 일반적으로 개발자가 하루에 의미 있는 PR을 1 2개 만드는 것이 보통이에요. 하네스가 루프를 자동화하니까 사람은 방향만 잡고, 반복 작업은 에이전트가 해요.

설계 철학

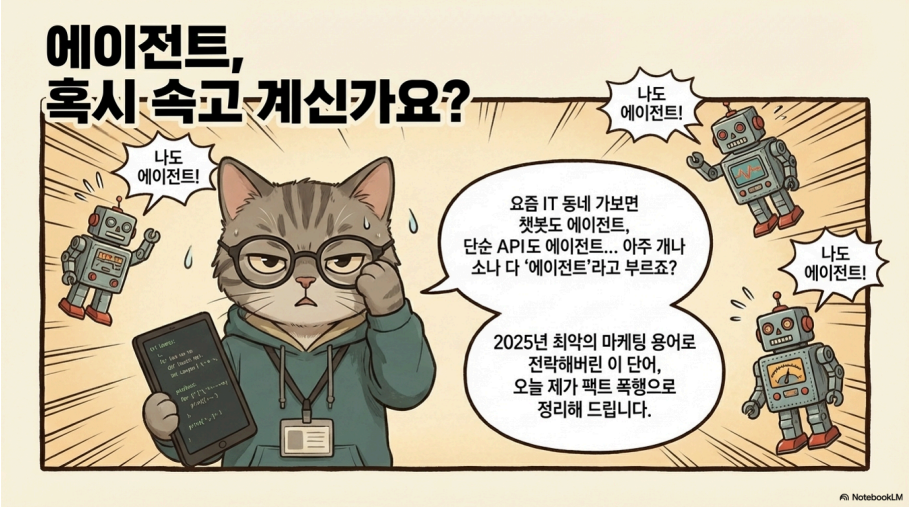
이 숫자들 뒤에는 하나의 설계 철학이 있어요. **비즈니스 로직을 최소화하고, 모델이 대부분의 일을 한다. 새 모델이 나올 때마다 코드를 지운다.**

이것이 의미하는 바는 명확해요. 좋은 하네스는 모델을 바꿔도 제품이 유지돼요. 나쁜 하네스는 모델을 바꾸면 전체를 다시 써야 해요. 하네스는 **인프라**여야 해요. 특정 모델에 종속된 로직이 들어가면 안 돼요.

> Claude Code의 핵심 설계 원칙: 비즈니스 로직을 최소화하고, 모델이 결정하게 하고, 새 모델이 나올 때마다 코드를 지워요. 좋은 하네스는 모델이 바뀌어도 유지되는 인프라예요.

9. 반패턴 3가지: 이렇게 하면 망해요

하네스를 설계할 때 빠지기 쉬운 세 가지 함정이 있어요.



[도표 4-3] 에이전트 현실 점검. 워크플로우와 에이전트를 구분하고, 에이전트에는 자동 검증이 반드시 필요해요.

반패턴 1: 워크플로우를 에이전트라고 부르기

입력을 받아 요약하고, 번역하고, 포맷팅하는 세 단계가 if-else로 고정돼 있다면 이것은 에이전트가 아니라 **워크플로우**예요.

Anthropic의 정의가 명확해요. **워크플로우는 LLM과 도구가 사전 정의된 코드 경로로 오케스트레이션되는 시스템이고, 에이전트는 LLM이 자신의 프로세스와 도구 사용을 동적으로 지휘하는 시스템이에요.** 워크플로우는 경로가 코드에 박혀 있고, 에이전트는 경로가 모델의 추론에 의해 런타임에 결정돼요.

비유하면, 워크플로우는 자판기예요. 버튼을 누르면 정해진 음료가 나와요. 에이전트는 바리스타예요. 손님의 취향을 물어보고, 재료를 확인하고, 상황에 맞게 만들어요. 자판기가 나쁜 것은 아니에요. 정해진 음료를 빠르게 뽑을 때는 자판기가 훨씬 좋아요. 문제는 자판기를 바리스타라고 부르면서 바리스타 수준의 유연성을 기대하는 거예요.

반패턴 2: 비즈니스 로직을 하네스에 직접 넣기

Python 파일이면 Black을 돌리고, TypeScript 파일이면 Prettier를 돌린다는 분기를 하네스 코드에 직접 넣으면 어떻게 될까요? 새 언어가 추가될 때마다 하네스 코드를 수정해야 해요. 열 가지 언어를 지원하면 열 개의 분기가 생기고, 모델을 바꾸면 전부 다시 확인해야 해요.

이런 규칙은 CLAUDE.md나 설정 파일에 넣어야 해요. 하네스는 인프라여야 하고, 비즈니스 로직은 설정에 있어야 해요. 앞서 본 Claude Code의 설계 철학 **새 모델이 나올 때마다 코드를 지운다**가 이 원칙의 실천이에요.

반패턴 3: 에이전트가 테스트를 수정하게 허용하기

이것이 가장 치명적인 반패턴이에요. AI가 **테스트를 수정해서 통과시켰습니다**라고 하면 아무것도 검증되지 않은 거예요. 에이전트가 테스트를 지우거나 수정해서 **통과**시키면, 검증 루프의 기반 자체가 사라져요.

Justin Young(Anthropic, 2025)의 명시적 경고가 있어요: **에이전트가 테스트를 지우거나 수정하게 허용하는 것은 용납할 수 없다.**

비유하면 이래요. 시험에서 답이 틀렸는데, 답안지를 고치는 대신 **채점 기준을 고쳐서** 맞다고 표시하는 거예요. 점수는 올라가지만 실력은 변하지 않았어요. 더 나쁜 것은, 이후로는 틀린 답도 다 맞다고 나오게 되니까 실력이 나빠지고 있는지조차 알 수 없게 돼요.

반패턴 3가지 정리

- **워크플로우를 에이전트라 부르기**: 고정된 경로와 동적 판단을 구분해야 해요
- **비즈니스 로직을 하네스에 넣기**: 하네스는 인프라, 규칙은 설정 파일에

- **테스트 수정 허용**: 채점 기준을 고치면 검증이 무의미해져요

10. Before/After: 하네스가 만드는 차이 총정리

이 차시에서 배운 모든 내용을 하나의 표로 정리해볼게요. 같은 모델, 같은 작업을 하네스 없이 했을 때와 있을 때의 차이예요.

작업	하네스 없이 (채팅)	하네스 있게 (에이전트)
버그 수정	사용자가 코드를 복사해 붙여넣고, 모델의 수정안을 다시 복사해 파일에 붙여넣고, 직접 테스트를 돌려요. 실패하면 에러 로그를 다시 복사해 대화에 붙여넣어요	사용자가 **로그인 버그 고쳐줘** 한 마디. 에이전트가 파일 탐색, 수정, 테스트, 재수정을 자동으로 반복해요
리팩토링	파일마다 하나씩 보여주고, 수정안을 받아 적용하고, 다음 파일을 보여줘요. 10개 파일이면 20번의 복사-붙여넣기	에이전트가 프로젝트 전체를 탐색해 관련 파일을 찾고, 의존성을 분석해 순서대로 수정하고, 전체 테스트를 돌려 확인해요
새 기능 개발	사용자가 요구사항을 설명하고, 모델이 코드를 생성하면 사용자가 파일에 넣고 실행해봐요. 오류가 나면 다시 대화	사용자가 요구사항을 설명하면 에이전트가 파일 생성, 라우트 추가, 테스트 작성, 실행, 수정까지 자동으로 진행해요
장기 작업 (3시간+)	불가능. 대화가 길어지면 모델이 앞부분을 잊어요. 사람이 매번 맥락을 다시 설명해야 해요	git commit과 progress 파일로 세션을 넘겨요. 서브에이전트로 컨텍스트를 분리해요

차이의 본질은 **누가 루프를 도는가**예요. 하네스 없이는 사람이 수집-행동-검증 루프의 모든 단계를 직접 실행해요. 하네스가 있으면 소프트웨어가 루프를 돌고 사람은 시작 명령만 내려요.

같은 버그 수정에 채팅에서는 30분이 걸리고 에이전트에서는 3분이 걸리는 이유가 여기에 있어요. 모델의 추론 속도가 빨라진 것이 아니에요. **사람이 병목이던 구간이 자동화된 것**이에요.

> 하네스의 핵심 가치: 사람이 수동으로 돌리던 수집-행동-검증 루프를 소프트웨어가 자동으로 돌려요. 같은 모델이라도 하네스가 있으면 30분이 3분으로 줄어드는 이유는, 사람이라는 병목이 제거되기 때문이에요.

이번 차시 핵심 정리

1. **Product Overhang**: 모델 능력은 이미 충분한데 그것을 꺼내 쓸 소프트웨어가 없었어요. 하네스가 그 소프트웨어예요.
2. **하네스의 두 역할**: 보이지 않는 프롬프트 관리(맥락 붙이기) + 호출 가능한 도구 제공(외부 세계 접근).
3. **CLAUDE.md**: 에이전트의 업무 매뉴얼. 빌드/테스트, 코드 스타일, 디렉토리 구조, 커밋 규칙 네 가지가 핵심.
4. **에이전트 루프**: Gather(수집) - Act(행동) - Verify(검증) - Repeat(반복). 검증이 빠지면 치명적.
5. **ACI 설계**: 도구를 처음 보는 사람처럼 명확하고, 실수하기 어렵게 설계. 프롬프트보다 도구 설명이 중요.
6. **검증 3단계**: 규칙 기반 - 시각 피드백 - LLM-as-Judge. 싼 것부터 적용.
7. **서브에이전트**: 컨텍스트 소진 문제를 별도 창으로 분리해 해결.
8. **반패턴**: 워크플로우를 에이전트라 부르기, 하네스에 비즈니스 로직 넣기, 테스트 수정 허용하기.

핵심 용어

용어	설명
하네스 (Harness)	LLM을 보이지 않는 프롬프트와 호출 가능한 도구로 확장하는 소프트웨어. 채팅봇을 에이전트로 바꾸는 핵심 요소
Product Overhang	모델 능력은 충분한데 그것을 제품으로 만드는 소프트웨어가 없는 상태
CLAUDE.md	프로젝트 루트에 놓는 에이전트용 설정 파일. 매 턴마다 자동으로 시스템 프롬프트에 주입됨

용어	설명
에이전트 루프	Gather(수집) - Act(행동) - Verify(검증) - Repeat(반복)의 반복 순환 패턴
ACI	Agent-Computer Interface. 에이전트가 도구를 사용하는 인터페이스를 설계하는 원칙
LLM-as-Judge	다른 LLM을 판정자로 사용해 에이전트의 결과물을 검증하는 방법
서브에이전트	메인 에이전트의 컨텍스트 소진을 막기 위해 별도 컨텍스트에서 실행되는 하위 에이전트
ReAct	추론 흔적(reasoning trace)과 행동(action)을 교차시키는 에이전트 루프의 원형 패턴
무상태 (Stateless)	LLM이 매 실행마다 이전 기억 없이 백지에서 시작하는 특성
워크플로우 vs 에이전트	워크플로우는 경로가 코드에 고정. 에이전트는 경로가 모델의 추론으로 결정

확인 문제

Q1. Claude.ai와 Claude Code에 같은 모델이 들어 있는데 결과가 다른 이유는 무엇인가요?

A. Claude Code에는 더 똑똑한 모델이 들어 있어서
B. Claude Code에는 하네스가 있어서 파일 접근, 도구 호출, 검증 루프가 가능하기 때문
C. Claude.ai는 인터넷에 연결되어 있지 않아서
D. Claude Code는 유료 서비스라서 성능이 더 좋음

힌트: Product Overhang의 핵심은 모델이 아니라 소프트웨어의 차이예요

Q2. 에이전트 루프의 네 단계를 순서대로 나열하고, 이 중 가장 자주 빠뜨려지면서 가장 치명적인 단계가 무엇인지 설명해보세요.

힌트: 의사의 진료 과정과 비교해서 생각해보세요. 약을 처방하고 나서 빠뜨리기 쉬운 것은?

Q3. 다음 중 ACI 설계의 올바른 원칙이 아닌 것은?

A. 도구 설명을 최대한 짧게 줄여서 컨텍스트를 절약한다
B. 상대 경로 대신 절대 경로를 강제한다
C. 모델이 도구를 처음 보는 주니어 개발자라고 상상하고 설계한다
D. 실수가 어렵도록 위험한 기본값을 제거한다

힌트: 도구 설명은 길어도 괜찮아요. 짧지만 모호한 것보다 길지만 명확한 것이 나아요

Q4. 반패턴 3가지 중 하나를 골라, 왜 그것이 위험한지 자신만의 비유로 설명해보세요.

힌트: 자판기와 바리스타, 채점 기준 변경 등의 비유를 참고하되 자신만의 비유를 만들어보세요

>> 다음 차시 예고

실습 4차시: 나만의 하네스 설계하기

CLAUDE.md를 직접 작성하고, 에이전트 루프가 실제로 돌아가는 과정을 체험해요. 같은 작업을 채팅과 에이전트로 각각 수행하면서 차이를 직접 확인해요.